

Line Editor in C: Submission

Team: Nikhi, Poornima, Pavana

Course: Portfolio Building – Studio Course, 3rd Semester Coding Competition

GitHub repository: <https://github.com/nikhi-sys/Portfolio/tree/main/Activity-3/Line-Editor>

Contents of the folder: line_editor.c, HELP.md, README.md, paper design photos (paper-design-1, paper-design-2)

Included in this document

1. Paper design (2 pages)
2. Compile and run
3. Command list (help)
4. Core features in use (append, insert, display, delete, count, quit)
5. GitHub commit history showing all three team members

Paper Design – Page 1

LINE EDITOR

Design:

- Data Structure: Dynamic array of strings ($\text{char}^{**}\text{lines}, \text{count}, \text{capacity}$).

Why: $\text{lines}[n-1]$ is line n , so access is fast. It grows with realloc .

Limit: Insert & delete shift lines (fine for small files).

Commands:

i n text	insert
d n	delete
w file	save
s word	search
h help	q quit

a test	append
p	print
l file	load
c	count

Functions:- insert_line, delete_line, display_document, save_file, load_file, main. (loop: read command, call function)

Core Logic:-

insert_line(n, text):

if $n < 1$ or $n > \text{count} + 1 \rightarrow \text{error, return}$.

grow array if full.

for $i = \text{count}$ down to n : $\text{lines}[i] = \text{lines}[i-1]$

$\text{lines}[n-1] = \text{copy of text}$

$\text{count}++$

delete_line(n):

if $\text{count} == 0 \rightarrow \text{"empty"}, \text{return}$.

if $n < 1$ or $n > \text{count} \rightarrow \text{error, return}$.

Paper Design – Page 2

```
free lines[n-1]
for i=n-1 to count-2: lines[i] = lines[i+1]
count--
```

```
display-document():
if count==0 → "empty"
for i=0 to count-1: print i+1, lines[i]
```

Eg:-

Insert at 2:

A	B	C
---	---	---

 →

A	X	B	C
---	---	---	---

Delete line 2:

A	X	B	C
---	---	---	---

 →

A	B	C
---	---	---

→ Edge Cases:-

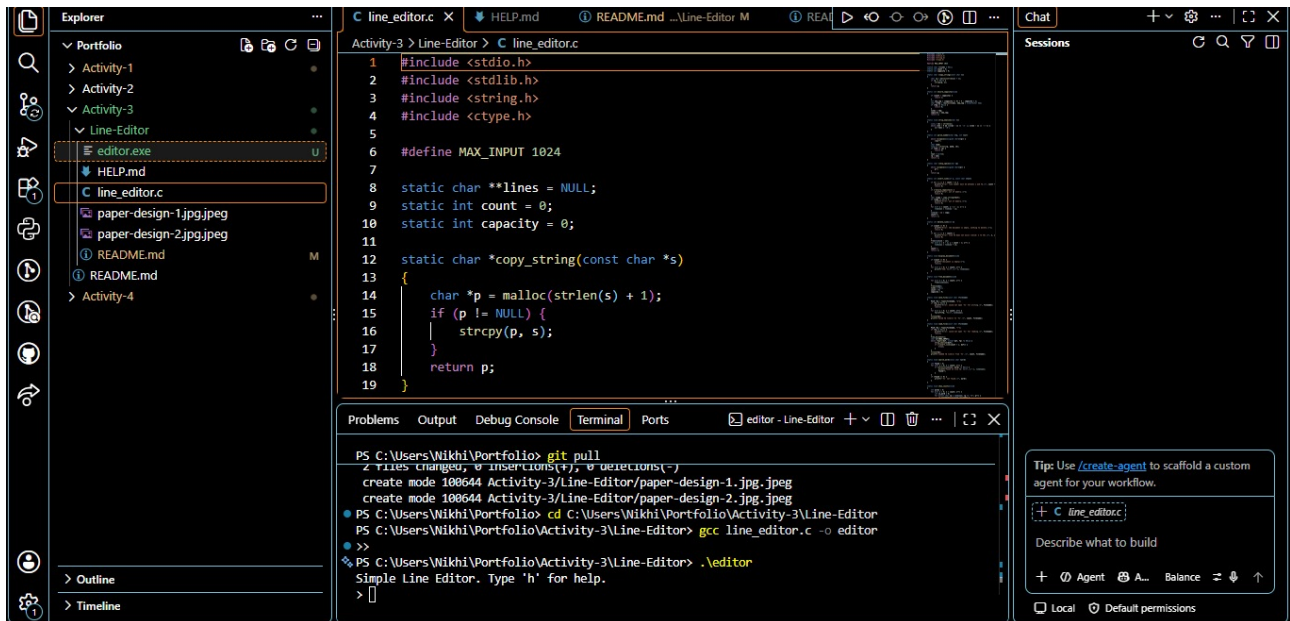
Empty doc, line 0, line too large, delete only line,
unknown command.

→ Bonus features:-

① Search (s)

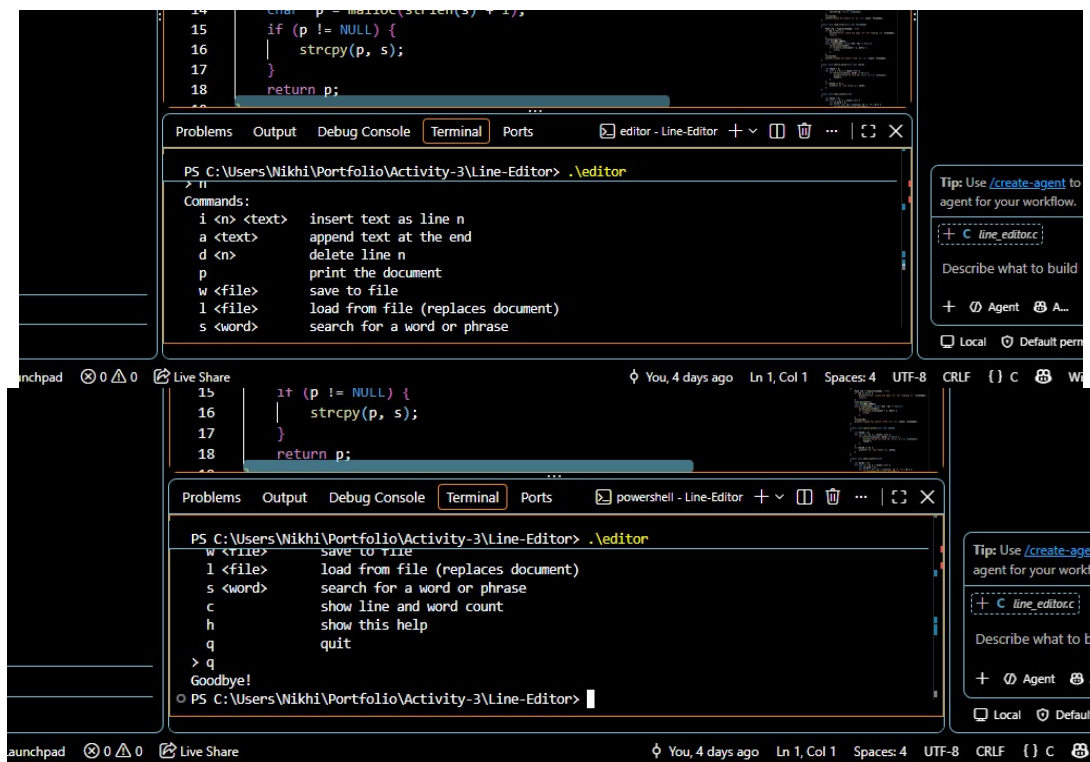
② Line count / word count (c)

1. Compile and run



Project folder in VS Code, compiled with `gcc line_editor.c -o editor` and started with `.\editor`. The editor prompt appears.

2. Command list (help)



Typing `h` lists all commands: insert, append, delete, print, save, load, search, count, help and quit. Typing `q` exits with "Goodbye!".

3. Core features in use



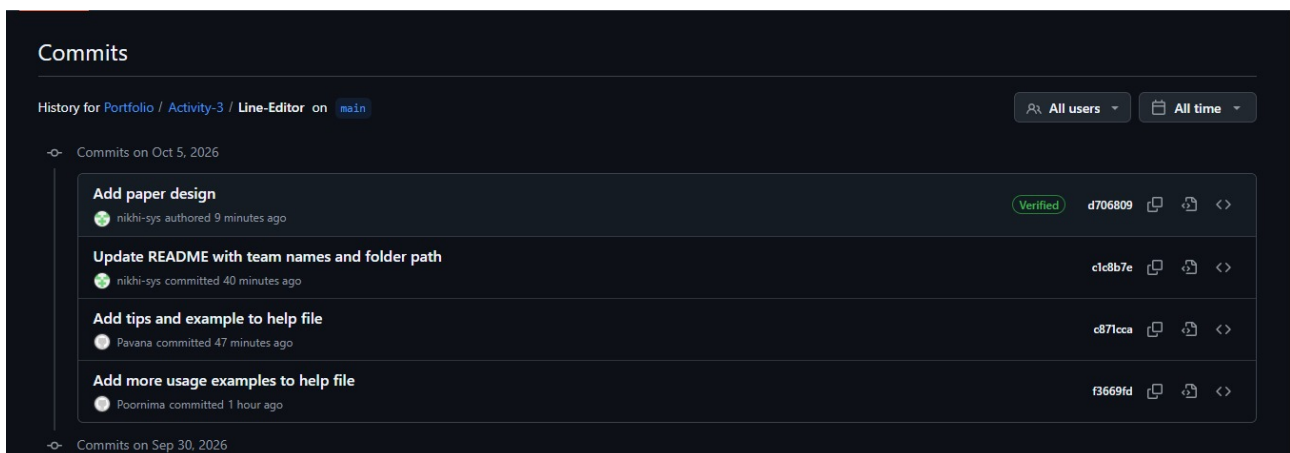
The first screenshot shows the terminal output of the Line-Editor. The user has compiled the program and run it. The editor displays a simple interface with a prompt. The user enters 'a Hello world', which is appended as line 1. Then, 'a This is my second line' is entered and appended as line 2. Next, 'i 1 My first line' is entered and inserted at line 1. Finally, 'p' is entered, and the document is printed. The second screenshot shows the user deleting line 2. The output shows 'Deleted line 2.' and the remaining lines are printed: '1: My first line' and '2: This is my second line'. The count shows 2 lines and 8 words.

```
PS C:\Users\Wikhi\Portfolio\Activity-3\Line-Editor> gcc line_editor.c -o editor
>> .\editor
Simple Line Editor. Type 'h' for help.
> a Hello world
Appended as line 1.
> a This is my second line
Appended as line 2.
> i 1 My first line
Inserted at line 1.
> p

PS C:\Users\Wikhi\Portfolio\Activity-3\Line-Editor> gcc line_editor.c -o editor
>> .\editor
> a 2
Deleted line 2.
> p
1: My first line
2: This is my second line
> c
Lines: 2, Words: 8
> q
```

Two lines are appended, a third is inserted at line 1, and the document is printed. Line 2 is then deleted, the remaining lines are printed (1: My first line, 2: This is my second line), and the count shows 2 lines and 8 words.

4. GitHub commit history



History of the Activity-3/Line-Editor folder showing commits from all three team members: nikhi-sys, Pavana and Poornima.